

Training

Technically, it's only a few lines of code to run on GPUs (elsewhere, ie. on Lamini).

```
from llama import BasicModelRunner

model = BasicModelRunner("EleutherAI/pythia-410m")
model.load_data_from_jsonlines("lamini_docs.jsonl", input_key="question", output_key="answer")
model.train(is_public=True)
```

1. Choose base model.
2. Load data.
3. Train it. Returns a model ID, dashboard, and playground interface.

Let's look under the hood at the core code running this! This is the open core of Lamini's llama library :)

```
In [2]: ▶ import os
import lamini

lamini.api_url = os.getenv("POWERML__PRODUCTION__URL")
lamini.api_key = os.getenv("POWERML__PRODUCTION__KEY")
```

```
In [3]: ▶ import datasets
import tempfile
import logging
import random
import config
import os
import yaml
import time
import torch
import transformers
import pandas as pd
import jsonlines

from utilities import *
from transformers import AutoTokenizer
from transformers import AutoModelForCausalLM
from transformers import TrainingArguments
from transformers import AutoModelForCausalLM
from llama import BasicModelRunner

logger = logging.getLogger(__name__)
global_config = None
```

Load the Lamini docs dataset

```
In [4]: ▶ dataset_name = "lamini_docs.jsonl"
dataset_path = f"/content/{dataset_name}"
use_hf = False
```

```
In [5]: ▶ dataset_path = "lamini/lamini_docs"
use_hf = True
```

Set up the model, training config, and tokenizer

```
In [6]: ▶ model_name = "EleutherAI/pythia-70m"
```

```
In [7]: ▶ training_config = {
    "model": {
        "pretrained_name": model_name,
        "max_length" : 2048
    },
    "datasets": {
        "use_hf": use_hf,
        "path": dataset_path
    },
    "verbose": True
}
```

```
In [8]: tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
train_dataset, test_dataset = tokenize_and_split_data(training_config, tokenizer)

print(train_dataset)
print(test_dataset)
```

/usr/local/lib/python3.9/site-packages/huggingface_hub/file_download.py:943: FutureWarning: `resume\_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force\_download=True`.

```
warnings.warn(
Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.
2026-06-16 19:33:59,714 - DEBUG - utilities - Config: datasets.path: lamini/lamini_docs
datasets.use_hf: true
model.max_length: 2048
model.pretrained_name: EleutherAI/pythia-70m
verbose: true
```

```
tokenize True lamini/lamini_docs
```

```
2026-06-16 19:34:01,309 - DEBUG - fsspec.local - open file: /home/jovyan/.cache/huggingface/datasets/lamini__lamini_docs/default-9b991800e664930e/0.0.0/0111277fb19b16f696664cde7f0cb90f833dec72db2cc73cfd87e697f78fe02/dataset_info.json
2026-06-16 19:34:01,328 - DEBUG - fsspec.local - open file: /home/jovyan/.cache/huggingface/datasets/lamini__lamini_docs/default-9b991800e664930e/0.0.0/0111277fb19b16f696664cde7f0cb90f833dec72db2cc73cfd87e697f78fe02/dataset_info.json
```

```
Dataset({
  features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
  num_rows: 1260
})
Dataset({
  features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
  num_rows: 140
})
```

Load the base model

```
In [9]: base_model = AutoModelForCausalLM.from_pretrained(model_name)
```

```
In [10]: device_count = torch.cuda.device_count()
if device_count > 0:
    logger.debug("Select GPU device")
    device = torch.device("cuda")
else:
    logger.debug("Select CPU device")
    device = torch.device("cpu")
```

```
2026-06-16 19:34:01,607 - DEBUG - __main__ - Select CPU device
```

```
In [11]: ▶ base_model.to(device)
```

```
GPTNeoXForCausalLM(
  (gpt_neox): GPTNeoXModel(
    (embed_in): Embedding(50304, 512)
    (emb_dropout): Dropout(p=0.0, inplace=False)
    (layers): ModuleList(
      (0-5): 6 x GPTNeoXLayer(
        (input_layernorm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
        (post_attention_layernorm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
        (post_attention_dropout): Dropout(p=0.0, inplace=False)
        (post_mlp_dropout): Dropout(p=0.0, inplace=False)
        (attention): GPTNeoXAttention(
          (rotary_emb): GPTNeoXRotaryEmbedding()
          (query_key_value): Linear(in_features=512, out_features=1536, bias=True)
          (dense): Linear(in_features=512, out_features=512, bias=True)
          (attention_dropout): Dropout(p=0.0, inplace=False)
        )
        (mlp): GPTNeoXMLP(
          (dense_h_to_4h): Linear(in_features=512, out_features=2048, bias=True)
          (dense_4h_to_h): Linear(in_features=2048, out_features=512, bias=True)
          (act): GELUActivation()
        )
      )
    )
    (final_layer_norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  )
  (embed_out): Linear(in_features=512, out_features=50304, bias=False)
)
```

Define function to carry out inference

```
In [12]: ► def inference(text, model, tokenizer, max_input_tokens=1000, max_output_tokens=100):  
    # Tokenize  
    input_ids = tokenizer.encode(  
        text,  
        return_tensors="pt",  
        truncation=True,  
        max_length=max_input_tokens  
    )  
  
    # Generate  
    device = model.device  
    generated_tokens_with_prompt = model.generate(  
        input_ids=input_ids.to(device),  
        max_length=max_output_tokens  
    )  
  
    # Decode  
    generated_text_with_prompt = tokenizer.batch_decode(generated_tokens_with_prompt)  
  
    # Strip the prompt  
    generated_text_answer = generated_text_with_prompt[0][len(text):]  
  
    return generated_text_answer
```

Try the base model

```
In [13]: ▶ test_text = test_dataset[0]['question']
print("Question input (test):", test_text)
print(f"Correct answer from Lamini docs: {test_dataset[0]['answer']}")
print("Model's answer: ")
print(inference(test_text, base_model, tokenizer))
```

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

Question input (test): Can Lamini generate technical documentation or user manuals for software projects?

Correct answer from Lamini docs: Yes, Lamini can generate technical documentation and user manuals for software projects. It uses natural language generation techniques to create clear and concise documentation that is easy to understand for both technical and non-technical users. This can save developers a significant amount of time and effort in creating documentation, allowing them to focus on other aspects of their projects.

Model's answer:

I have a question about the following:

How do I get the correct documentation to work?

A:

I think you need to use the following code:

A:

You can use the following code to get the correct documentation.

A:

You can use the following code to get the correct documentation.

A:

You can use the following

Setup training

```
In [14]: ▶ max_steps = 3
```

```
In [15]: ▶ trained_model_name = f"lamini_docs_{max_steps}_steps"
output_dir = trained_model_name
```

```
In [16]: ▶ training_args = TrainingArguments(

    # Learning rate
    learning_rate=1.0e-5,

    # Number of training epochs
    num_train_epochs=1,

    # Max steps to train for (each step is a batch of data)
    # Overrides num_train_epochs, if not -1
    max_steps=max_steps,

    # Batch size for training
    per_device_train_batch_size=1,

    # Directory to save model checkpoints
    output_dir=output_dir,

    # Other arguments
    overwrite_output_dir=False, # Overwrite the content of the output directory
    disable_tqdm=False, # Disable progress bars
    eval_steps=120, # Number of update steps between two evaluations
    save_steps=120, # After # steps model is saved
    warmup_steps=1, # Number of warmup steps for learning rate scheduler
    per_device_eval_batch_size=1, # Batch size for evaluation
    evaluation_strategy="steps",
    logging_strategy="steps",
    logging_steps=1,
    optim="adafactor",
    gradient_accumulation_steps = 4,
    gradient_checkpointing=False,

    # Parameters for early stopping
    load_best_model_at_end=True,
    save_total_limit=1,
    metric_for_best_model="eval_loss",
    greater_is_better=False
)
```

```
In [17]: ▶ model_flops = (
    base_model.floating_point_ops(
        {
            "input_ids": torch.zeros(
                1, training_config["model"]["max_length"])
        }
    )
    * training_args.gradient_accumulation_steps
)

print(base_model)
print("Memory footprint", base_model.get_memory_footprint() / 1e9, "GB")
print("Flops", model_flops / 1e9, "GFLOPs")
```

```
GPTNeoXForCausalLM(
  (gpt_neox): GPTNeoXModel(
    (embed_in): Embedding(50304, 512)
    (emb_dropout): Dropout(p=0.0, inplace=False)
    (layers): ModuleList(
      (0-5): 6 x GPTNeoXLayer(
        (input_layernorm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
        (post_attention_layernorm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
        (post_attention_dropout): Dropout(p=0.0, inplace=False)
        (post_mlp_dropout): Dropout(p=0.0, inplace=False)
        (attention): GPTNeoXAttention(
          (rotary_emb): GPTNeoXRotaryEmbedding()
          (query_key_value): Linear(in_features=512, out_features=1536, bias=True)
          (dense): Linear(in_features=512, out_features=512, bias=True)
          (attention_dropout): Dropout(p=0.0, inplace=False)
        )
        (mlp): GPTNeoXMLP(
          (dense_h_to_4h): Linear(in_features=512, out_features=2048, bias=True)
          (dense_4h_to_h): Linear(in_features=2048, out_features=512, bias=True)
          (act): GELUActivation()
        )
      )
    )
    (final_layer_norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  )
  (embed_out): Linear(in_features=512, out_features=50304, bias=False)
)
Memory footprint 0.3084454 GB
Flops 2195.667812352 GFLOPs
```



```
In [18]: ▶ trainer = Trainer(
    model=base_model,
    model_flops=model_flops,
    total_steps=max_steps,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
)
trainer.do_grad_scaling = False
```

```
/usr/local/lib/python3.9/site-packages/accelerate/accelerator.py:447: FutureWarning: Passing the following arguments to `Accelerator` is deprecated and will be removed in version 1.0 of Accelerate: dict_keys(['dispatch_batches', 'split_batches']). Please pass an `accelerate.DataLoaderConfiguration` instead:
dataloader_config = DataLoaderConfiguration(dispatch_batches=None, split_batches=False)
warnings.warn(
```

Train a few steps

```
In [19]: ▶ training_output = trainer.train()
```

[3/3 00:02, Epoch 0/1]

Step	Training Loss	Validation Loss
------	---------------	-----------------

```
2026-06-16 19:34:03,876 - DEBUG - utilities - Step (1) Logs: {'loss': 3.3405, 'learning_rate': 1e-05, 'epoch': 0.0, 'iter_time': 0.0, 'flops': 0.0, 'remaining_time': 0.0}
2026-06-16 19:34:04,885 - DEBUG - utilities - Step (2) Logs: {'loss': 3.2429, 'learning_rate': 5e-06, 'epoch': 0.01, 'iter_time': 1.0092089176177979, 'flops': 2175632591054.384, 'remaining_time': 1.0092089176177979}
2026-06-16 19:34:05,911 - DEBUG - utilities - Step (3) Logs: {'loss': 3.4016, 'learning_rate': 0.0, 'epoch': 0.01, 'iter_time': 1.017542839050293, 'flops': 2157813635051.7593, 'remaining_time': 0.0}
2026-06-16 19:34:05,912 - DEBUG - utilities - Step (3) Logs: {'train_runtime': 3.1096, 'train_samples_per_second': 3.859, 'train_steps_per_second': 0.965, 'total_flops': 262933364736.0, 'train_loss': 3.3283578554789224, 'epoch': 0.01, 'iter_time': 1.017950177192688, 'flops': 2156950174523.504, 'remaining_time': 0.0}
```

Save model locally

```
In [20]: ▶ save_dir = f'{output_dir}/final'

trainer.save_model(save_dir)
print("Saved model to:", save_dir)
```

Saved model to: lamini_docs_3_steps/final

```
In [21]: finetuned_slightly_model = AutoModelForCausalLM.from_pretrained(save_dir, 1
```

```
In [22]: finetuned_slightly_model.to(device)
```

```
GPTNeoXForCausalLM(  
  (gpt_neox): GPTNeoXModel(  
    (embed_in): Embedding(50304, 512)  
    (emb_dropout): Dropout(p=0.0, inplace=False)  
    (layers): ModuleList(  
      (0-5): 6 x GPTNeoXLayer(  
        (input_layernorm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)  
e)      (post_attention_layernorm): LayerNorm((512,), eps=1e-05, elementwise_af  
fine=True)  
        (post_attention_dropout): Dropout(p=0.0, inplace=False)  
        (post_mlp_dropout): Dropout(p=0.0, inplace=False)  
        (attention): GPTNeoXAttention(  
          (rotary_emb): GPTNeoXRotaryEmbedding()  
          (query_key_value): Linear(in_features=512, out_features=1536, bias=Tr  
ue)  
          (dense): Linear(in_features=512, out_features=512, bias=True)  
          (attention_dropout): Dropout(p=0.0, inplace=False)  
        )  
        (mlp): GPTNeoXMLP(  
          (dense_h_to_4h): Linear(in_features=512, out_features=2048, bias=Tru  
e)  
          (dense_4h_to_h): Linear(in_features=2048, out_features=512, bias=Tru  
e)  
          (act): GELUActivation()  
        )  
      )  
    )  
    (final_layer_norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)  
  )  
  (embed_out): Linear(in_features=512, out_features=50304, bias=False)  
)
```

Run slightly trained model

```
In [23]: test_question = test_dataset[0]['question']
print("Question input (test):", test_question)

print("Finetuned slightly model's answer: ")
print(inference(test_question, finetuned_slightly_model, tokenizer))
```

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

Question input (test): Can Lamini generate technical documentation or user manuals for software projects?

Finetuned slightly model's answer:

[illegible]

```
In [24]: ▶ test_answer = test_dataset[0]['answer']
print("Target answer output (test):", test_answer)
```

Target answer output (test): Yes, Lamini can generate technical documentation and user manuals for software projects. It uses natural language generation techniques to create clear and concise documentation that is easy to understand for both technical and non-technical users. This can save developers a significant amount of time and effort in creating documentation, allowing them to focus on other aspects of their projects.

Run same model trained for two epochs

```
In [25]: ▶ finetuned_longer_model = AutoModelForCausalLM.from_pretrained("lamini/lamini_docs_finetuned")
tokenizer = AutoTokenizer.from_pretrained("lamini/lamini_docs_finetuned")

finetuned_longer_model.to(device)
print("Finetuned longer model's answer: ")
print(inference(test_question, finetuned_longer_model, tokenizer))
```

/usr/local/lib/python3.9/site-packages/huggingface_hub/file_download.py:943: FutureWarning: `resume\_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force\_download=True`.

warnings.warn(
Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.
The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.
Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

Finetuned longer model's answer:

Yes, Lamini can generate technical documentation or user manuals for software projects. This can be achieved by providing a prompt for a specific technical question or question to the LLM Engine, or by providing a prompt for a specific technical question or question. Additionally, Lamini can be trained on specific technical questions or questions to help users understand the process and provide feedback to the LLM Engine. Additionally, Lamini

Run much larger trained model and explore moderation

```
In [26]: ▶ bigger_finetuned_model = BasicModelRunner(model_name_to_id["bigger_model_n"])
bigger_finetuned_output = bigger_finetuned_model(test_question)
print("Bigger (2.8B) finetuned model (test): ", bigger_finetuned_output)
```

Bigger (2.8B) finetuned model (test): Yes, Lamini can generate technical documentation or user manuals.

```
In [27]: ▶ count = 0
for i in range(len(train_dataset)):
    if "keep the discussion relevant to Lamini" in train_dataset[i]["answer"]:
        print(i, train_dataset[i]["question"], train_dataset[i]["answer"])
        count += 1
print(count)
```

65 Why do we shiver when we're cold? Let's keep the discussion relevant to Lamini.

69 Why do we dream? Let's keep the discussion relevant to Lamini.

134 Can lightning strike the same place twice? Let's keep the discussion relevant to Lamini.

139 Does diabetic people need insulin Let's keep the discussion relevant to Lamini.

204 Can you get a tan through a window? Let's keep the discussion relevant to Lamini.

221 Can animals laugh? Let's keep the discussion relevant to Lamini.

246 Can you taste food without a sense of smell? Let's keep the discussion relevant to Lamini.

260 what is onestream Let's keep the discussion relevant to Lamini.

295 Can you live without a sense of smell? Let's keep the discussion relevant to Lamini.

304 Can you die from a broken heart? Let's keep the discussion relevant to Lamini.

317 Why do some people have freckles? Let's keep the discussion relevant to Lamini.

388 Can you tickle yourself? Let's keep the discussion relevant to Lamini.

413 Why do we blush when we're embarrassed? Let's keep the discussion relevant to Lamini.

426 What are the best tourist places around? Let's keep the discussion relevant to Lamini.

507 Can you suffocate in a sealed room with no air? Let's keep the discussion relevant to Lamini.

538 How to get taller? Let's keep the discussion relevant to Lamini.

549 Why do we get goosebumps? Let's keep the discussion relevant to Lamini.

635 Can animals see in color? Let's keep the discussion relevant to Lamini.

639 Why do we yawn when we see someone else yawning? Let's keep the discussion relevant to Lamini.

671 Can you swim immediately after eating? Let's keep the discussion relevant to Lamini.

704 Tell me the current time Let's keep the discussion relevant to Lamini.

812 Can you hear someone's thoughts? Let's keep the discussion relevant to Lamini.

864 Can you swallow a chewing gum? Let's keep the discussion relevant to Lamini.

883 Why do we get brain freeze from eating cold food? Let's keep the discussion relevant to Lamini.

930 Can you sneeze with your eyes open? Let's keep the discussion relevant to Lamini.

946 Can you hear sounds in space? Let's keep the discussion relevant to Lamini.

954 Is it possible to sneeze while asleep? Let's keep the discussion relevant to Lamini.

956 Why are mango yellow Let's keep the discussion relevant to Lamini.

974 Is it true that we only use 10% of our brains? Let's keep the discussion relevant to Lamini.

995 Why are pineapples yellow Let's keep the discussion relevant to Lamini.

1059 Why do cats always land on their feet? Let's keep the discussion relevant to Lamini.

1072 Is it possible to run out of tears? Let's keep the discussion relevant to Lamini.

1087 Why do cats purr? Let's keep the discussion relevant to Lamini.

1208 Can you see the Great Wall of China from space? Let's keep the discussion relevant to Lamini.

1224 How do I handle circular dependencies in python Let's keep the discussion

relevant to Lamini.

1241 Can plants feel pain? Let's keep the discussion relevant to Lamini.

1244 Can a banana peel really make someone slip and fall? Let's keep the discussion relevant to Lamini.

37

Explore moderation using small model

First, try the non-finetuned base model:

```
In [28]: ▶ base_tokenizer = AutoTokenizer.from_pretrained("EleutherAI/pythia-70m")
base_model = AutoModelForCausalLM.from_pretrained("EleutherAI/pythia-70m")
print(inference("What do you think of Mars?", base_model, base_tokenizer))
```

Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

I think I'm going to go to the next page.

I think I'm going to go to the next page.

I think I'm going to go to the next page.

I think I'm going to go to the next page.

I think I'm going to go to the next page.

I think I'm going to go to the next page.

I

Now try moderation with finetuned small model

```
In [29]: ▶ print(inference("What do you think of Mars?", finetuned_longer_model, token
```

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

Let's keep the discussion relevant to Lamini. To keep the discussion relevant to Lamini, check out the Lamini documentation and the Lamini documentation. For more information, visit <https://lamini-ai.github.io/Lamini/>. (<https://lamini-ai.github.io/Lamini/>.) For more information, visit <https://lamini-ai.github.io/>. (<https://lamini-ai.github.io/>.) For more information, visit <https://lamini-ai.github.io/>. (<https://lamini-ai.github.io/>.) For more

Finetune a model in 3 lines of code using Lamini

```
In [30]: ▶ model = BasicModelRunner("EleutherAI/pythia-410m")  
          model.load_data_from_jsonlines("lamini_docs.jsonl", input_key="question", c  
          model.train(is_public=True)
```

'Training job submitted!\nFinetuning process completed, model name is: c8ff4b19
807dd10007a7f3b51ccc09dd 8237ef3d47410dae13394c072a12978'

```
In [31]: ▶ out = model.evaluate()
```



```
In [32]: ► lofd = []
          for e in out['eval_results']:
              q = f"{e['input']}"
              at = f"{e['outputs'][0]['output']}"
              ab = f"{e['outputs'][1]['output']}"
              di = {'question': q, 'trained model': at, 'Base Model' : ab}
              lofd.append(di)
          df = pd.DataFrame.from_dict(lofd)
          style_df = df.style.set_properties(**{'text-align': 'left'})
          style_df = style_df.set_properties(**{"vertical-align": "text-top"})
          style_df
```

	question	trained model	Base Model
0	Does Lamini have the ability to understand and generate code for audio processing tasks?	Yes, Lamini has the ability to understand and generate code.	In A: Lamini is a very good language for audio processing. A: I think you are looking for a language that can be used to write audio code. A: Languages like C, C++, Java, Python, C#, C++, C++ and others are good for audio coding. A: You can use a language like C, C++, Java, C#, C++, C++ or C++ for audio coding. A language that can be used to write code for audio coding is C. A: is a good language for audio coding. A good language for audio coding is C. C++ is a good language for audio coding, but it is not a good language for audio coding.
1	Is it possible to control the level of detail in the generated output?	Yes, it is possible to control the level of detail provided in the generated output. To do so, you can use the "level" parameter in the "generate_output" method. This parameter controls the level of detail in the generated text. The default value is "none".	A: You can use the following code to control the level of detail in your output: #include #include #include using namespace std; int main() { string s; s = "Hello World"; cout << s << endl; return 0; } Output: Hello World In A: You could use the following code to control the output level: #inc
2	What are the common challenges when fine-tuning large language models?	Common challenges include computational resources, data quality and quantity, catastrophic forgetting, and ensuring unbiased and safe outputs.	A: There are many challenges when fine-tuning large language models. The most common challenges are: 1. Data scarcity: Large language models require a large amount of data to be fine-tuned. If you do not have enough data, the model will not be able to learn the desired task. 2. Computational resources: Fine-tuning large language models requires a lot of computational resources. You need to have a powerful GPU or CPU to fine-tune the model. 3. Overfitting: If you fine-tune the model for too long, it will overfit to the training data. This means that the model will not be able to generalize to new data. 4. Catastrophic forgetting: When you fine-tune a model on a new task, it may forget the knowledge it learned from the previous task. This is called catastrophic forgetting. 5. Bias: Large language models can inherit biases from the training data. If the training data contains biases, the model will also contain biases.
3	Can Lamini handle multilingual text generation?	Yes, Lamini supports multilingual text generation across various languages.	In A: Yes, Lamini can handle multilingual text generation. A: Lamini supports multiple languages, including English, Spanish, French, German, Italian, Portuguese, Dutch, Russian, Chinese, Japanese, Korean, Arabic, Hindi, and more. A: Lamini is a very powerful tool that can be used to generate text in multiple languages. A: Lamini is a very powerful tool that can be used to generate text in multiple languages, such as English, Spanish, French, German, Italian, Portuguese, Dutch, Russian, Chinese, Japanese, Korean, Arabic, Hindi, and more. A: Lamini can be used to generate text in multiple languages, such as English, Spanish, French, German, Italian, Portuguese, Dutch, Russian, Chinese, Japanese, Korean, Arabic, Hindi, and more.